

Can we pick training data so that a circuit forms early?

How circuits form in small transformers, and how to select data for them

Research summary · Luca Columbu · 2026 · Full draft and code: <https://github.com/lucacolumbu/LLM-research>

What we wanted to know

A language model learns its abilities as small internal *circuits*: groups of attention heads and neurons that together perform one computation. We wanted to understand how one such circuit forms during training, which properties of the training data make it form sooner or later, and whether we could then *choose* training data so that the circuit forms earlier, or with less data, than on a random sample of the same corpus.

Today’s data-selection methods (filtering by loss, matching a target distribution, removing duplicates) score each document with one number that mixes every ability together. They cannot say which ability a decision helps or hurts, because they never look at what the model builds inside. We wanted a score aimed at one specific circuit.

The circuit we chose, and the trick

We chose the *induction head*, the best-understood circuit in transformers and the one behind in-context learning. Its job is simple: when the model sees a token it has seen before in the current context, it looks back to that earlier occurrence and predicts whatever followed it last time. Given “Mr Dursley ... Mr”, it predicts “Dursley”.

This is also exactly what the `gzip` compressor does: its LZ77 algorithm finds the earlier occurrence of the current string and replaces the repeat with a pointer to it. So `gzip` can stand in for the circuit. Our score for each document is its *compression gain*: how much shorter it compresses than a shuffled copy of itself. The shuffle keeps the word counts, so only genuine repeated structure counts. The score is free to compute and needs no model.

How we tested it

1. **Built synthetic tasks with adjustable knobs.** Random token sequences with an embedded repeat, where we control how much is repeated, how long the copies are, how noisy the data is and how big the vocabulary is. Each setting is a training set with a different compression gain.
2. **Trained small transformers and recorded when the circuit forms.** Checkpoints every 100 steps; at each one we measured whether the induction head exists (a head attending to the right earlier token, and patching its output into a corrupted run restoring the answer) and whether the model copies correctly. The formation step is where these cross a threshold. Three seeds per setting, about 200 runs.
3. **Checked the prediction.** Does a dataset’s compression gain tell us, before training, when the circuit will form?
4. **Used it to select data.** From a mixed pool, kept a fixed-size subset by gain and compared it with a random subset of the same size and with an oracle that sees the hidden generating parameters.
5. **Repeated the selection on real data.** 2.5 billion tokens of Python from The Stack, on a rented GPU.

What we found

- **The score predicts when the circuit forms.** Across the synthetic settings, higher gain means earlier formation, almost perfectly in rank (Spearman -0.98 attention-only, -0.92 with MLPs, -1.0 on condition averages). The circuit arrives as a sudden jump, and its timing follows the score rather than the amount of repetition we had dialled in.
- **Selecting by the score forms the circuit much earlier with a third of the data.** From a pool of 60,000 documents, the 20,000 with the highest gain form the circuit at step 700 in every seed; a random 20,000 at step 5,200 or never. The score even beat the oracle (800–1,000): the oracle knows how much repetition was *requested* per document, the compressor measures how much was *produced*.
- **The data decides which version of the circuit forms.** We expected one circuit and found a family. Depending on how long the copies are, the model builds an induction head that looks k tokens behind the ideal position, which caps its accuracy. Curves that looked like partial learning were finished circuits of the wrong variant, grown on top of a simpler positional circuit the model builds first.
- **Where the score fails, a small model can take over.** With a tiny vocabulary, accidental repeats fool the

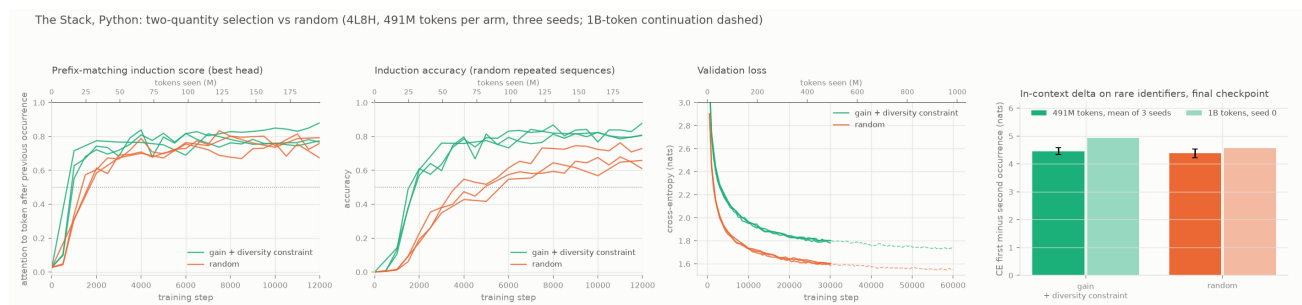
compressor. A small model that already has an induction head restores the prediction: the compressor is a hand-written version of the circuit, a trained model a learned one.

- **Repetition within a document and variety across documents are different things.** A second score, the compression *distance* between documents, measures variety. It does not predict formation but relates to how well a second task’s circuit generalises to unseen combinations. The two scores pull in opposite directions when selecting.

On real code

From 2.5B tokens of Python (9.77M windows of 256 tokens) we selected a fifth by gain, with a variety constraint that skips near-duplicate boilerplate, and compared it with a random fifth. Same 4-layer model, same 491M training tokens, three seeds each; one seed per arm continued to 983M.

Training data	Head forms (step)	Copies correctly (step)	Final copy accuracy	Validation loss
Selected by score	1,000, every seed	2,000, every seed	0.84–0.86	1.78–1.80
Random	1,500–2,000	4,000–6,000	0.70–0.78	1.59–1.61



The selected data forms the induction head 1.5 to 2 times sooner and the copying behaviour 2 to 3 times sooner, in every seed, and the model keeps copying more accurately to the end. The cost is in the last column: the model trained on selected data is a worse general language model by 0.2 nats, because the most repetitive fifth of a corpus is a narrower slice of it, and doubling the training budget does not close the gap. Once both models have the circuit, a test on rare variable names (how much cheaper the second occurrence is than the first) shows they use it equally well. Selection by this score buys *when* the circuit forms; the second score, variety, is what limits the price. A smaller study with four selection strategies showed the trade-off can be tuned: gain buys the circuit, variety buys the distribution.

We also learned to tell in advance where the method cannot work. On children’s stories, long repeated passages cover under 0.4% of tokens, the score has nothing to grip, and no induction head formed in any run. The compressor’s own match statistics predicted that before we trained anything.

What this involved

Everything was built from scratch in Python with PyTorch and TransformerLens: task generators, training and checkpointing, the circuit measurements (attention patterns, exact logit attribution, activation patching, minimal sufficient head sets), the scores, the selector, and a streaming pipeline for The Stack. One habit paid off: trace what a trained model actually does before running a sweep. Three times this caught a model solving the task by a shortcut that would have given clean results about the wrong circuit, and the task was redesigned each time. Almost all runs fit on a laptop; the real-data study cost about \$7 of rented GPU time.

What is still open

Models are small (2 to 4 layers), one circuit family is covered, and the formation steps are early compared with production budgets. We have not shown selection working on prose, and the natural next step is a circuit where the compressor should fail and a learned score take over: facts stated in several paraphrases.